PROJECT REPORT

Title:

“BANGALORE HOUSE PRICE PREDICTION SYSTEM

USING MACHINE LEARNING AND FASTAPI”

Submitted By

Name: Nandita **Department:** Computer Science and Engineering

Roll No: 20241CSE0635 **College:** Presidency University Bangalore

Under the Guidance of

Guide Name:

Shaikh Faisal,

Winger IT Solutions-Software Development Branch,

Bangalore

INDEX

ABSTRACT

The **Bangalore House Price Prediction System** is a machine learning-based web application designed to estimate house prices based on different property characteristics.

Real estate price estimation is an important task because property values depend on multiple factors such as location, area, facilities, and market conditions.

This project uses machine learning techniques to analyze historical housing data and predict the approximate price of a house.

The system allows users to enter details such as:

- Total square feet
- Number of bathrooms
- Price per square feet
- Location

The backend of the application is developed using FastAPI, which provides API services for communication between the user interface and the machine learning model.

The trained machine learning model is stored using Pickle and loaded into the FastAPI application during execution.

The frontend interface is created using HTML, CSS, and JavaScript to provide a simple and interactive user experience.

The application also stores previous predictions for future reference.

The system is made accessible outside the local network using ngrok, allowing users to access the application from different devices.

This project shows the practical use of Machine Learning in the real estate industry by combining data processing, model prediction, API development, and web deployment.

INTRODUCTION, OBJECTIVES AND SCOPE

2.1 Introduction

The real estate industry generates large amounts of data related to properties, locations, and prices.

Finding the correct value of a house manually is difficult because prices vary according to many factors.

Machine learning helps in solving this problem by learning patterns from previous housing data and predicting future values.

This project develops a web-based application where users can enter property details and receive an estimated price instantly.

The system combines:

- Machine Learning
- FastAPI Backend
- HTML Frontend
- API Communication

2.2 Objectives

The main objectives of this project are:

- To build a machine learning model for predicting house prices.
- To analyze housing data and identify important factors.
- To preprocess and clean the dataset.
- To develop a FastAPI-based prediction API.
- To create an interactive frontend application.
- To connect frontend and backend.
- To provide real-time price prediction.
- To store prediction history.
- To make the application accessible online.

2.3 Scope

The project can be useful for:

- Property buyers
- Property sellers
- Real estate companies
- Market researchers

The system helps users estimate property prices quickly.

The project can be extended by adding:

- More property features
- Database storage
- Advanced machine learning algorithms
- Mobile application

SYSTEM ANALYSIS AND METHODOLOGY

3.1 System Analysis

The system analyzes housing data and predicts prices using machine learning.

The complete process includes:

1. Collecting dataset
2. Cleaning data
3. Selecting features
4. Training model
5. Saving model
6. Integrating model with FastAPI
7. Creating frontend
8. Generating prediction

3.2 Dataset Description

The dataset contains Bangalore house information.

Important features:

Location

Represents the area where the property is located.

Example:

Indiranagar
Whitefield
Electronic City

Total Square Feet

Represents the total area of the house.

Bathroom

Represents the number of bathrooms.

Price Per Square Feet

Represents the cost of one square feet area.

Price

The final house price value.

3.3 Data Processing

Before training the model, the dataset is processed.

Data Cleaning

Unwanted values and incorrect records are removed.

Missing Value Handling

Missing values are checked and handled.

Duplicate Removal

Duplicate records are removed.

Feature Selection

Important columns are selected.

Selected features:

total_sqft
bath
price_per_sqft
location

3.4 Feature Encoding

Machine learning models cannot understand text directly.

Location values are converted into numerical values.

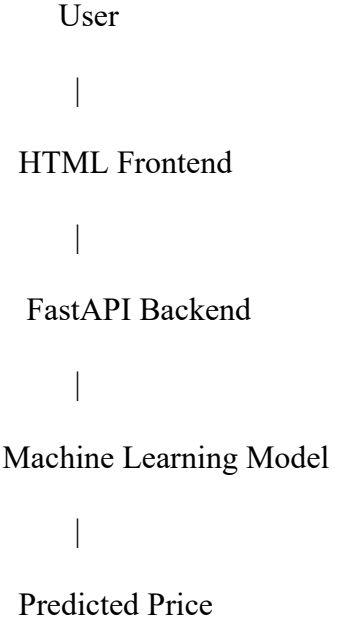
Example:

Indiranagar

location_Indiranagar = 1

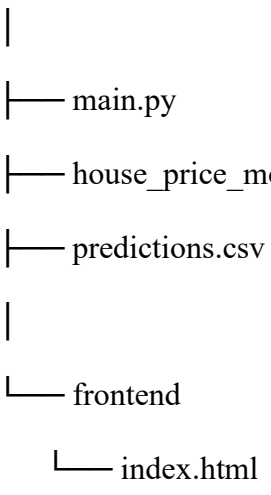
SYSTEM DESIGN AND IMPLEMENTATION

4.1 System Architecture



4.2 Project Structure

House Prediction Project



4.3 Backend Implementation

The backend is developed using FastAPI.

Main responsibilities:

- Loading trained model
- Receiving user data
- Processing input
- Generating prediction
- Returning result

Code:

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse

import pickle
import numpy as np
import pandas as pd
import csv
import os
from datetime import datetime
```

```
# ================== APP ==================
app = FastAPI(
    title="Bangalore House Price Prediction API",
    version="1.0"
)
```

```
# ================== FRONTEND ==================
if os.path.exists("frontend"):
    app.mount("/frontend", StaticFiles(directory="frontend"), name="frontend")
```

```
@app.get("/home")
def frontend():
    return FileResponse("frontend/index.html")
```

```
# ================== LOAD MODEL ==================
with open("house_price_model.pkl", "rb") as file:
    model = pickle.load(file)

print("Model loaded successfully")
```

```
# ================== INPUT MODEL ==================
from pydantic import BaseModel

class HouseData(BaseModel):
    total_sqft: float
    bath: int
    price_per_sqft: float
    location: str
```

```
# ================== ROOT ==================
@app.get("/")
def home():
    return {"message": "API Running Successfully"}
```

```python
# ================= LOCATIONS =================
@app.get("/locations")
def locations():
    data = [
        col.replace("location_", "")
        for col in model.feature_names_in_
        if col.startswith("location_")
    ]
    return {"locations": data}


# ================= PREDICT =================
@app.post("/predict")
def predict(data: HouseData):

    columns = model.feature_names_in_

    input_data = pd.DataFrame(
        np.zeros((1, len(columns))),
        columns=columns
    )

    # numeric features
    input_data["total_sqft"] = data.total_sqft
    input_data["bath"] = data.bath
    input_data["price_per_sqft"] = data.price_per_sqft

    # location encoding
    location_col = "location_" + data.location

    if location_col in columns:
        input_data[location_col] = 1
    else:
        return {"error": "Location not found"}

    # prediction
    result = model.predict(input_data)
    price = round(float(result[0]), 2)

    if price <= 0:
        return {"error": "Invalid prediction"}

    # save history
    file_exists = os.path.exists("predictions.csv")

    with open("predictions.csv", "a", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)

        if not file_exists:
            writer.writerow([
                "date",
                "total_sqft",
                "bath",
                "price_per_sqft",
                "location",
                "predicted_price_lakhs"
```

```
            ])

        writer.writerow([
            datetime.now(),
            data.total_sqft,
            data.bath,
            data.price_per_sqft,
            data.location,
            price
        ])

    return {
        "location": data.location,
        "predicted_price_lakhs": price,
        "status": "saved"
    }


# ================ HISTORY ================
@app.get("/history")
def history():

    if not os.path.exists("predictions.csv"):
        return {"total_searches": 0, "history": []}

    df = pd.read_csv("predictions.csv")

    return {
        "total_searches": len(df),
        "history": df.to_dict(orient="records")
    }
```

4.4 Machine Learning Model

Algorithm used:

Linear Regression

Linear Regression predicts house prices by identifying relationships between input features and output values.

Input:

total_sqft
bath
price_per_sqft
location

Output:

Predicted Price

The trained model is saved as:

house_price_model.pkl

4.5 Frontend Implementation

The frontend is developed using:

- HTML
- CSS
- JavaScript

Features:

- User input form
- Location selection
- Prediction button
- Result display

Code:

```
<!DOCTYPE html>
<html>
<head>
<title>ML House Price Dashboard</title>

<style>
```

```
/* ================== BASE ================== */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: 'Segoe UI', sans-serif;
}
```

```
body {
  background: linear-gradient(135deg, #dbeafe, #fce7f3);
  padding: 20px;
}
```

```
/* ================== CONTAINER ================== */
.container {
  max-width: 900px;
  margin: auto;
}
```

```
/* ================== TITLE ================== */
h1 {
  text-align: center;
  margin-bottom: 20px;
  font-size: 30px;
  font-weight: 800;
  color: #1e3a8a;
}
```

```
/* ================== MAIN BOX ================== */
.main-box {
  background: white;
```

```
    padding: 25px;
    border-radius: 20px;
    box-shadow: 0 12px 30px rgba(0,0,0,0.08);
    text-align: center;
}

/* ================== INPUTS ================== */
.inputs {
    display: flex;
    flex-direction: column;
    gap: 12px;
    margin-top: 10px;
}

input, select {
    padding: 12px;
    border-radius: 12px;
    border: 1px solid #c7d2fe;
    outline: none;
    font-size: 14px;
    background: #f8fafc;
}

input:focus, select:focus {
    border-color: #60a5fa;
    box-shadow: 0 0 8px rgba(96,165,250,0.4);
}

/* ================== BUTTONS ================== */
.btn-group {
    margin-top: 15px;
}

button {
    padding: 12px 18px;
    border: none;
    border-radius: 14px;
    cursor: pointer;
    font-weight: bold;
    transition: 0.3s;
}

/* Predict Button */
.predict-btn {
    background: linear-gradient(135deg, #93c5fd, #60a5fa);
    color: white;
}

/* History Button */
.history-btn {
    background: linear-gradient(135deg, #fbcfe8, #f9a8d4);
    color: #831843;
    margin-left: 10px;
}
```

```css
/* ================== RESULT ================== */
.result {
  margin-top: 15px;
  font-weight: bold;
  color: #2563eb;
}

/* ================== STATS ================== */
.stats {
  margin-top: 20px;
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 15px;
}

.stat-box {
  padding: 18px;
  border-radius: 18px;
  font-weight: bold;
  text-align: center;
  box-shadow: 0 8px 18px rgba(0,0,0,0.06);
}

/* Blue card */
.stat-blue {
  background: #dbeafe;
  border: 2px solid #93c5fd;
}

/* Pink card */
.stat-pink {
  background: #fce7f3;
  border: 2px solid #f9a8d4;
}

/* ================== HISTORY ================== */
#historyBox {
  display: none;
  margin-top: 20px;
}

/* ================== TABLE ================== */
table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 10px;
  border-radius: 14px;
  overflow: hidden;
}

th {
  background: #60a5fa;
  color: white;
  padding: 10px;
}
```

```
td {
  text-align: center;
  padding: 10px;
  border-bottom: 1px solid #eee;
}

tr:hover {
  background: #fce7f3;
}

</style>
</head>

<body>

<div class="container">

<h1>🏠House Price Dashboard</h1>

<!-- ================= MAIN BOX ================= -->
<div class="main-box">

  <h3>📊 Prediction Box</h3>

  <div class="inputs">
    <input id="sqft" placeholder="Total Sqft">
    <input id="bath" placeholder="Bath">
    <input id="price" placeholder="Price per Sqft">
    <select id="location"></select>
  </div>

  <div class="btn-group">
    <button class="predict-btn" onclick="predict()">Predict Price</button>
    <button class="history-btn" id="historyBtn" onclick="toggleHistory()">View History</button>
  </div>

  <div class="result" id="result"></div>

  <!-- ================= STATS ================= -->
  <div class="stats">

    <div class="stat-box stat-blue">
      📈 Total Searches<br>
      <span id="totalSearches">0</span>
    </div>

    <div class="stat-box stat-pink">
      💰 Last Prediction<br>
      <span id="lastPrice">₹0</span>
    </div>

  </div>

</div>
```

```
<!-- ================== HISTORY ================== -->
<div id="historyBox">

  <div class="main-box">

    <h3>📜 Prediction History</h3>

    <table>
      <thead>
        <tr>
          <th>Date</th>
          <th>Sqft</th>
          <th>Bath</th>
          <th>Price/Sqft</th>
          <th>Location</th>
          <th>Predicted Price</th>
        </tr>
      </thead>
      <tbody id="historyTable"></tbody>
    </table>

  </div>

</div>

</div>

<script>

let historyVisible = false;

// ================== LOAD LOCATIONS ==================
fetch("/locations")
.then(r => r.json())
.then(data => {
  const loc = document.getElementById("location");
  data.locations.forEach(l => {
    let opt = document.createElement("option");
    opt.value = l;
    opt.text = l;
    loc.appendChild(opt);
  });
});


// ================== PREDICT ==================
function predict() {

  const sqft = document.getElementById("sqft").value;
  const bath = document.getElementById("bath").value;
  const price = document.getElementById("price").value;
  const location = document.getElementById("location").value;

  if (!sqft || !bath || !price || !location) {
```

```
        document.getElementById("result").innerText = "Please fill all fields";
        document.getElementById("result").style.color = "red";
        return;
    }

    fetch("/predict", {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({
            total_sqft: +sqft,
            bath: +bath,
            price_per_sqft: +price,
            location: location
        })
    })
    .then(r => r.json())
    .then(res => {

        if (res.error) {
            document.getElementById("result").innerText = res.error;
            document.getElementById("result").style.color = "red";
            return;
        }

        document.getElementById("result").innerText =
            "Predicted Price: ₹ " + res.predicted_price_lakhs + " Lakhs";

        document.getElementById("lastPrice").innerText =
            "₹ " + res.predicted_price_lakhs;

        loadHistory();
    });
}


// ================= TOGGLE HISTORY =================
function toggleHistory() {

    const box = document.getElementById("historyBox");
    const btn = document.getElementById("historyBtn");

    historyVisible = !historyVisible;

    if (historyVisible) {
        box.style.display = "block";
        btn.innerText = "Hide History";
        loadHistory();
    } else {
        box.style.display = "none";
        btn.innerText = "View History";
    }
}


// ================= LOAD HISTORY =================
function loadHistory() {
```

```
  fetch("/history")
  .then(r => r.json())
  .then(data => {

      document.getElementById("totalSearches").innerText =
          data.total_searches || 0;

      const table = document.getElementById("historyTable");
      table.innerHTML = "";

      if (!data.history || data.history.length === 0) {
          table.innerHTML = `
              <tr>
                  <td colspan="6">No history available</td>
              </tr>
          `;
          return;
      }

      data.history.forEach(r => {
          table.innerHTML += `
              <tr>
                  <td>${r.date || "-"}</td>
                  <td>${r.total_sqft || "-"}</td>
                  <td>${r.bath || "-"}</td>
                  <td>${r.price_per_sqft || "-"}</td>
                  <td>${r.location || "-"}</td>
                  <td>₹ ${r.predicted_price_lakhs || "-"}</td>
              </tr>
          `;
      });
  });
}

</script>

</body>
</html>
```

TESTING, RESULTS AND DEPLOYMENT

5.1 API Testing

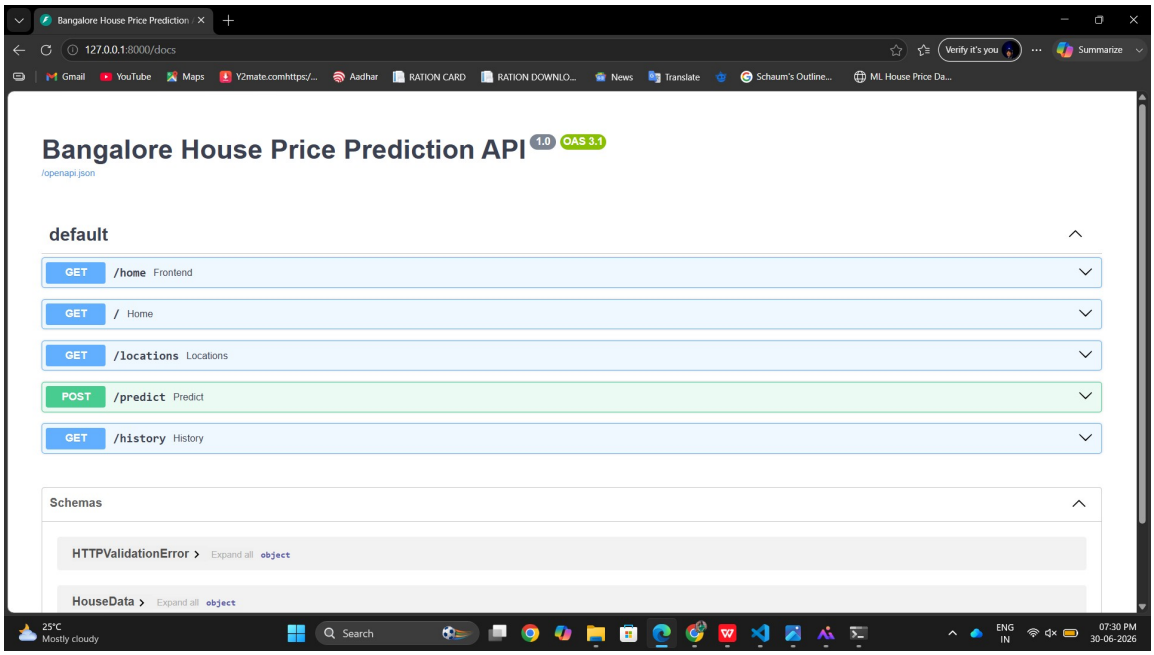
FastAPI provides automatic documentation.

Swagger page:

/docs

Example:

https://mummy-opossum-crane.ngrok-free.dev/docs



5.2 Prediction Testing

Example Input:

Total sqft: 1200

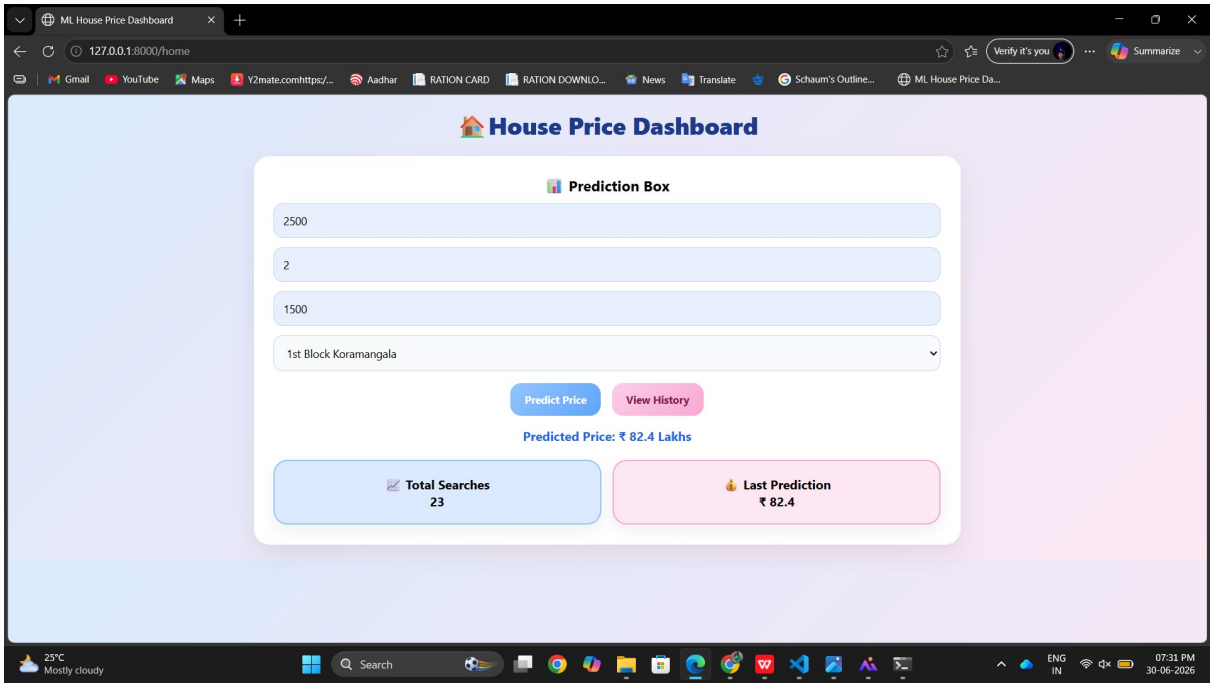
Bathroom: 2

Price per sqft: 5000

Location: Indiranagar

Output:

Predicted Price: 85.6 Lakhs

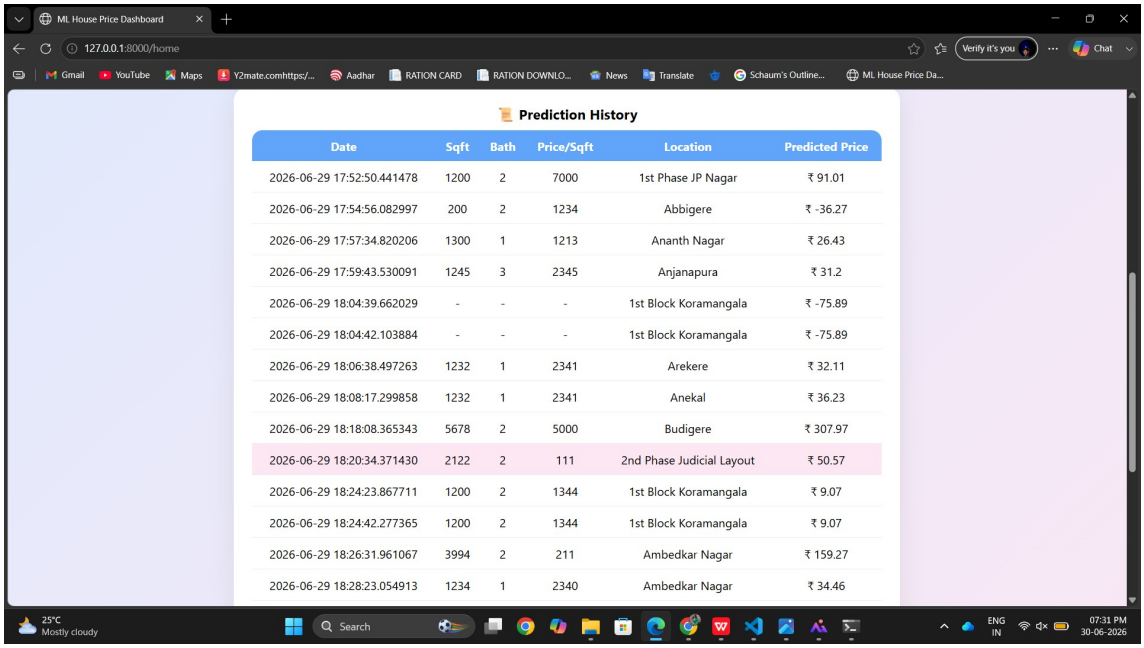


5.3 Prediction History

The system stores previous predictions.

Stored details:

- Date
- Area
- Bathroom
- Location
- Predicted price



5.4 Deployment Using ngrok

The application runs locally using FastAPI.

Local server:

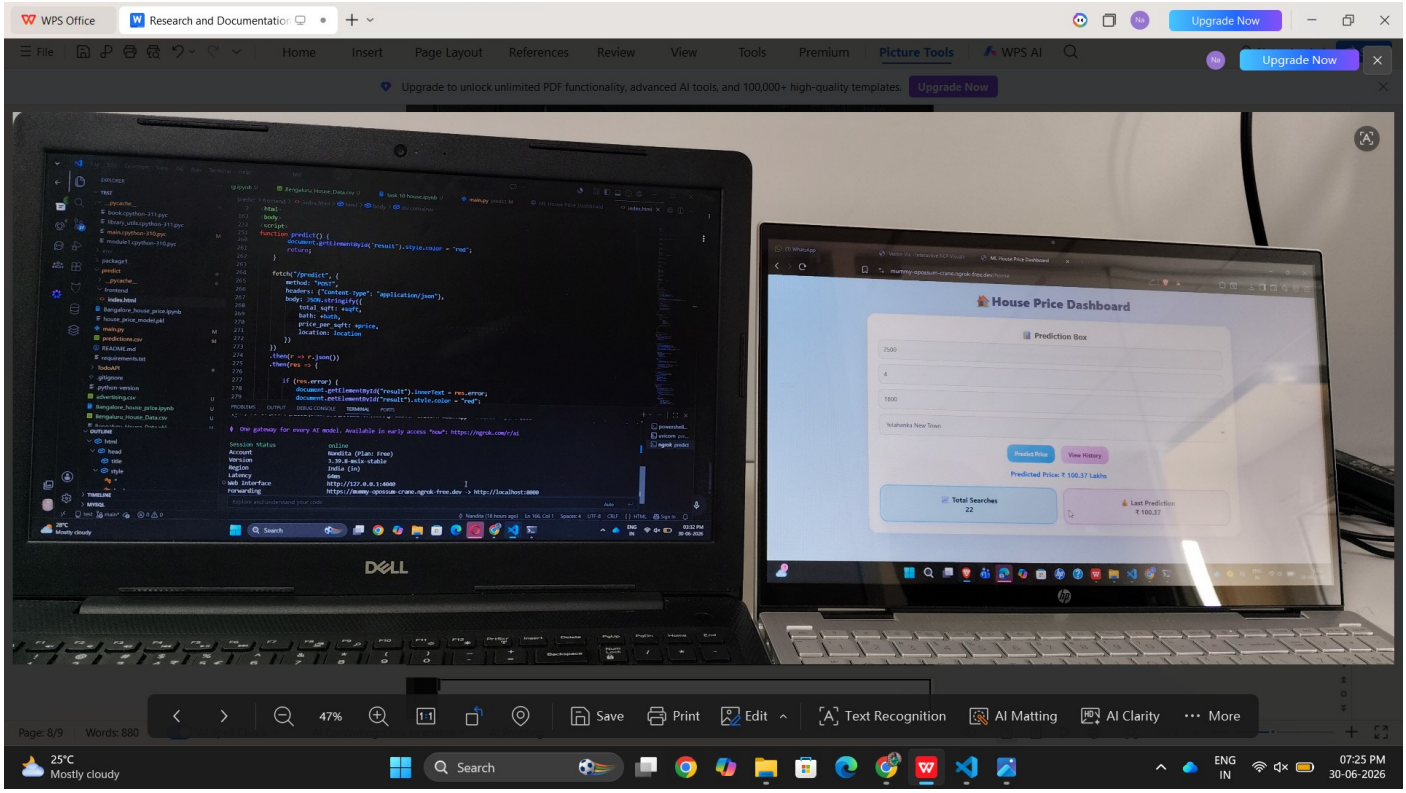
localhost:8000

ngrok provides public access:

https://mummy-opossum-crane.ngrok-free.dev

Users can access the application from:

- Laptop
- Mobile
- Other devices



CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Bangalore House Price Prediction System successfully combines Machine Learning and Web Development.

The application predicts house prices based on property information provided by users.

FastAPI provides a powerful backend system and connects the machine learning model with the frontend.

The project demonstrates how machine learning can be applied to real-world applications such as real estate price prediction.

6.2 Future Enhancements

Future improvements include:

- Permanent cloud deployment
- Database integration
- Improved user interface
- Mobile application development
- Advanced machine learning algorithms
- Adding more property details
- Data visualization dashboard
- User authentication system